Week 11

데이터 정제 · 변환 · 집계

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
df = pd.read_csv("students.csv", dtype={"student_id": str})
df.info(); df.isna().sum(); df["department"].value_counts()
df.loc[df["year"] == 1, "bonus"] = 5            # 체인 인덱싱 대신 loc
df[(df["final"] > 80) & (df["year"] >= 3)]      # and 대신 &
```

students.csv는 **깨끗한** 데이터였다 — 실제 데이터는?

오늘의 목표

1. 지저분한 데이터의 문제를 **진단해 정제 계획 표**로 만든다
2. `.str` · `replace` / `map` · `to_numeric` · `to_datetime` · `where`로 정제
3. 정제 단계를 **파이프라인 함수**로 묶고 전후 비교 · 테스트
4. `groupby().agg()` · `pivot_table` · `merge` · `cut`으로 집계
5. AI 정제 코드의 **순서 문제**와 **잘못된 결측 채우기**를 찾는다

수업 흐름

구간	시간	활동
도입 · 개념	20분	지저분한 데이터 진단, 정제 계획
안내 실습	45분	정제 도구, 파이프라인
독립 · 짝 실습	35분	집계: groupby · pivot · merge · cut
공유 · 점검	20분	AI 정제 코드 검증, 과제 안내

이번 주 Colab 실습 열기

설문 원본: survey_raw.csv

```
response_id,submitted_at,department,grade,coding_hours,uses_ai,satisfaction
R149,2025-09-08 08:48,컴공,2학년,0.5시간,yes,3
R009,2025/09/05 10:20,간호학과, 2 ,0.5,yes,5
R117,2025.09.11 03:40,경영,2,1,yes,1
```

열	표기 종류
department	14가지 (`컴공` , `보안학과` , `컴퓨터 공학과` …)
uses_ai	8가지 (`예` , `Y` , `yes` , `사용함` …)
coding_hours	`2시간` , `약 3` , 빈 칸, `30` , `-1`

먼저 원본 그대로 읽기

```
raw = pd.read_csv("survey_raw.csv", dtype=str, keep_default_na=False)
```

- `dtype=str` → 자동 변환으로 문제가 **가려지지 않게**
- `keep_default_na=False` → 빈 칸이 `""`로 남아 **정확히 셀 수 있게**

```
raw["department"].value_counts()
raw["coding_hours"].unique()
raw.duplicated().sum()            # 6
```

정제 계획 표

열	문제	방법
전체	중복 6건	`drop_duplicates`
department	공백 · 줄임말	공백 제거 → 사전으로 통일
grade	`"2학년"`, `" 2 "`	숫자 추출 → int
coding_hours	단위 · 빈 칸 · 30 · −1	숫자 추출 → 0~16 밖은 NaN
uses_ai	8가지 표기	True / False 매핑
satisfaction	`"4점"`, 빈 칸	숫자 추출 → `Int64`
submitted_at	`-` `/` `.`	`to_datetime(format="mixed")`

> 이 표가 그대로 **AI 프롬프트**가 된다

.str 메서드와 정규식

```
df["department"].str.strip().str.replace(" ", "")
df["grade"].str.extract(r"(\d)")[0].astype(int)
df["coding_hours"].str.extract(r"(-?\d+\.?\d*)")[0]
```

정규식	의미	예
`\d`	숫자 한 글자	`"2학년"` → `2`
`\d+`	숫자 여러 글자	`"30"`
`-?\d+\.?\d*`	음수 · 소수 포함	`"약 2.5"` → `2.5`
`\s+`	공백 여러 개	`"컴퓨터  공학과"`

replace vs map

```
alias = {"컴공": "컴퓨터공학과", "보안학과": "정보보안학과", "경영": "경영학과", ...}
s.replace(alias)          # 사전에 없는 값은 그대로
s.map(ai_map)             # 사전에 없는 값은 NaN
```

	사전에 없는 값	쓰임
`replace`	**그대로 둠**	일부만 고칠 때
`map`	**NaN**	모든 값을 바꿔야 할 때 → 빠뜨린 표기 발견

```
uses_ai.isna().sum()      # 0이어야 빠뜨린 표기가 없다
```

숫자 · 날짜 변환

```
hours = pd.to_numeric(hours_text, errors="coerce")   # 실패 → NaN
sat = pd.to_numeric(..., errors="coerce").astype("Int64")   # 결측 허용 정수
when = pd.to_datetime(df["submitted_at"], format="mixed")
when.dt.dayofweek, when.dt.hour, when.dt.date
```

- `Int64` (대문자 I): 결측이 있어도 **정수** 유지
- `format="mixed"` : 행마다 형식이 달라도 변환

이상값: where vs clip

```
hours.where(hours.between(0, 16))     # 범위 밖 → NaN
hours.clip(0, 16)                     # 범위 밖 → 경계값(0 또는 16)
```

원래 값	`where`	`clip`
30	NaN	16
−1	NaN	0
2.5	2.5	2.5

> 응답 **오류**라면 `where`(모름으로), 측정 **한계**라면 `clip` — 선택 이유를 보고서에

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- 진단: `value_counts`, `unique`, `duplicated`
- `.str` · 정규식 · `replace` / `map` · `to_numeric` · `to_datetime` · `where`

Level 2 바꾸기

- 사전에서 `"보안학과"`를 빼고 `map` → 무엇이 NaN이 되나
- 상한 16 → 10으로 바꾸면 결측 증가량
- `where` vs `clip` 결과 비교

실습 3 (Level 3): 정제 파이프라인

```
def clean_survey(raw_df):
    """설문 원본 → 정제된 DataFrame. 원본은 바꾸지 않는다."""
    df = raw_df.drop_duplicates().copy()
    out = pd.DataFrame({
        "department": df["department"].str.strip().str.replace(" ", "").replace(DEPT_ALIAS),
        "grade": df["grade"].str.extract(r"(\d)")[0].astype(int),
        "uses_ai": df["uses_ai"].str.strip().map(AI_MAP),
        ...
    })
    out["coding_hours"] = out["coding_hours"].where(out["coding_hours"].between(0, 16))
    return out
```

새 데이터가 와도 **같은 방식으로** → 재현성

정제 전후 비교표

항목	정제 전	정제 후
행 수	156	150
학과 표기 수	14	5
학년 표기 수	12	4
AI 사용 표기 수	8	2
코딩 시간 결측	12	15

결측이 **늘어난 이유**(범위 밖 3건 → NaN)를 반드시 설명

파이프라인 테스트

```
def test_clean_survey():
    s = clean_survey(raw)
    assert len(s) == 150 and s["response_id"].is_unique
    assert set(s["department"]) == {"간호학과", "경영학과", "전자공학과",
                                    "정보보안학과", "컴퓨터공학과"}
    assert s["uses_ai"].notna().all()                    # 매핑 누락 없음
    assert s["coding_hours"].dropna().between(0, 16).all()
    assert raw.shape == (156, 8)                         # 원본 불변
```

groupby().agg()

```
survey.groupby("department").agg(
    responses=("response_id", "size"),
    avg_hours=("coding_hours", "mean"),
    ai_rate=("uses_ai", "mean"),        # True 비율
    avg_sat=("satisfaction", "mean"),
).round(2)
```

`새 이름=("원래 열", "함수")` — `size`(전체 행) · `count`(결측 제외) · `mean` · `sum` · `max` · `nunique`

pivot_table과 crosstab

```
survey.pivot_table(index="grade", columns="uses_ai",
                   values="satisfaction", aggfunc="mean")

pd.crosstab(survey["grade"], survey["uses_ai"], margins=True)   # 개수
```

grade	AI 미사용	AI 사용
1	3.00	3.63
3	2.43	3.56

두 기준으로 **펼쳐서** 비교

merge: 다른 표와 합치기

```
depts = pd.read_csv("departments.csv")     # department, college, building
merged = survey.merge(depts, on="department", how="left",
                      validate="many_to_one")
assert len(merged) == len(survey)          # 행 수 확인!
merged.groupby("college").agg(...)
```

- `how="left"` : 왼쪽 표의 행을 모두 유지
- `validate="many_to_one"` : 오른쪽 키 **중복** 방지
- 키가 중복되면 행이 **몰래 늘어난다**

cut: 구간 나누기

```
survey["hours_band"] = pd.cut(
    survey["coding_hours"], bins=[0, 1, 3, 6, 16],
    labels=["1시간 이하", "1~3시간", "3~6시간", "6시간 초과"],
    include_lowest=True)

survey.groupby("hours_band", observed=True)["satisfaction"].mean()
```

연속된 숫자 → **범주**로 바꿔 그룹 비교

AI 함정 1: 정제 순서

```
messy = pd.DataFrame({"name": ["김민준", "김민준 ", "이서연"],
                      "dept": ["컴공", "컴퓨터공학과", "경영학과"]})
messy.drop_duplicates()            # 3행 — 같은 사람이 남음
```

표기를 통일한 **뒤에** 중복 제거 → 2행

> 순서: **표기 통일 → 중복 제거 → 형 변환 → 이상값 처리**

AI 함정 2: 결측 채우기

```
df["department"].fillna(df["department"].mean())    # TypeError
df["satisfaction"].fillna(df["satisfaction"].mean()) # 3.47 같은 없는 응답
```

열 성격	결측 처리
범주형 (학과)	`"미상"` 표시, 최빈값, 또는 그대로
순서형 (1~5 만족도)	그대로 두기 (평균 X)
연속형 (시간)	분석 목적에 따라 그대로 / 중앙값

AI에게 정제를 요청할 때

```
clean_survey(raw_df)를 작성해 주세요. 원본은 바꾸지 마세요.
1. department: 공백 제거 후 {"컴공": "컴퓨터공학과", ...}로 통일
2. 표기 통일 후 drop_duplicates
3. coding_hours: 숫자만 추출해 float, 0~16 밖은 NaN (clip 아님)
4. satisfaction: 숫자만 추출해 Int64, 결측은 채우지 않음
마지막에 행 수 · 고유값 · 범위를 확인하는 assert를 포함하세요.
```

정제 계획 표 = 프롬프트 → AI가 임의로 채우거나 자르는 일을 막는다

이번 주 과제 (10점)

문제	내용	배점
1	`normalize_dept` 학과명 표준화 (`"기타"` 포함)	2
2	`parse_hours` 숫자 추출 + 이상값 NaN	2
3	`clean_survey` 파이프라인 (요일 열, 정제 후 중복 제거)	3
4	`college_summary` · `sat_pivot` (AI 협업)	3

문제 4: merge 후 **행 수 확인**, 결측을 **임의로 채우지 않기**

자기 점검

1. 정제 전에 `dtype=str, keep_default_na=False`로 읽는 이유는?
2. `replace`와 `map`의 차이는?
3. 응답 오류로 보이는 `30시간`은 `where`와 `clip` 중 무엇으로?
4. merge 후 행 수가 늘어나는 원인은?
5. 만족도(1~5) 결측을 평균으로 채우면 안 되는 이유는?

정리와 다음 주

오늘	핵심
진단	정제 계획 표
정제	`.str` · 정규식 · `replace` / `map` · `to_numeric` · `where`
파이프라인	함수 + 전후 비교 + 테스트
집계	`agg` · `pivot_table` · `merge(validate)` · `cut`
AI	정제 순서, 열 성격에 맞는 결측 처리

다음 주 · Week 12 탐색적 데이터 분석과 프로젝트 기획